常驻、主动型智能体
如何工作

Lee Robinson 讲座 · 中文教学笔记

2026 年 10 月 13 日整理



视频作者/频道：Lee Robinson · Stanford CS146S
发布日期：授课：2026-10-08；X 发布：2026-10-09
视频时长：45 分 17 秒
视频链接：https://x.com/leerob/status/2108650243365736855

目录

1 从工具循环走向常驻智能体 4

1.1 问题的起点：模型能执行，不等于任务能持续 4
1.2 开场代码：工具结果为什么要回到消息列表 5
1.3 四个阶段：人的角色怎样变化 6
1.4 从等待与批准转向持续引导 6
1.5 聊天界面承载真实委托 7
1.6 本章小结 7

2 支撑长期协作的能力与文件体系 7

2.1 为什么现在可以尝试这种架构 7
2.2 子智能体如何减轻主上下文负担 8
2.3 四类文件解决四类问题 8
2.4 插件连接工具与知识 9
2.5 本章小结 9

3 常驻系统的架构：事件、状态与可靠执行 10

3.1 持续可用为何不需要持续占用计算机 10
3.2 事件入口、服务端与工具计算机各做什么 10
3.3 Firecracker 计算机与独立生命周期 12
3.4 发送消息之后：同步与去重 13
3.5 从任务队列走向持久工作流 14
3.6 用户如何打断后台工作 14
3.7 本章小结 15

4 Harness：工具发现、上下文分工与行动边界 15

4.1 一个沟通接口可以支持丰富交互 15
4.2 动态工具发现：先索引，再读取细节 16
4.3 先使用便宜而可靠的能力 17
4.4 理解网页：截图、HTML 与无障碍树 18
4.5 主智能体与 helpers：把执行细节留在局部 18
4.6 工具设计的三条实际经验 19
4.7 长期行动需要检查输入与授权边界 20
4.8 本章小结 20

5 上下文工程：让长期运行成为可持续的系统 21

5.1 为什么常驻智能体必须管理上下文 21

5.2 提示缓存：复用稳定前缀，降低重复输入成本 21

5.3 保护主智能体的上下文 22

5.4 缓存顺序、发布变更与预热 22

5.5 上下文压缩：容量、成本与信息损失 23

5.6 分层记忆：稳定事实、历史日志与临时便笺 24

5.7 本章小结 25

6 产品与模型如何一起进步 25

6.1 让模型熟悉自己的运行环境 25

6.2 前台回应与后台工作的不同节奏 26

6.3 内循环：把真实错误变成评测 26

6.4 外循环：训练新模型，解锁新产品行为 26

6.5 “Delete the product”：准备随着模型能力重构 27

6.6 本章小结 27

7 未来方向与开放问题 27

7.1 长期工作、记忆和技能学习 27

7.2 长任务与评测周期的矛盾 28

7.3 主动性的分寸：什么时候应该打扰用户 28

7.4 新的软件基础构件 29

7.5 本章小结 29

8 总结与延伸 30

8.1 回到问题：常驻、主动型智能体怎样工作 30

8.2 用一个任务贯穿所有模块 30

8.3 读懂一段最小教学伪代码 31

8.4 讲者最后给工程师的三条建议 31

8.5 进一步思考：在哪些地方衡量系统是否进步 32

8.6 拓展阅读 32

8.7 本章小结 32

阅读说明

本笔记依据 Lee Robinson 在 Stanford CS146S 的英文讲座编写。视频原题为 *How always-on agents work*，封面记录授课日期为 2026 年 10 月 8 日；所给 X 链接的发布日期为 10 月 9 日。正文使用平台提供的英文字幕，并结合原始幻灯片画面整理。

内容按教学关系重组，覆盖整个视频的实质内容。带时间脚注的插图来自原视频；标注“根据讲座整理”的图以及“补充解释”“教学伪代码”属于笔记作者为理解机制所作的表达，不是讲者公开的产品源码。模型效果与未来趋势的判断保留归属，示意数字不作为产品规格。

原视频定位	学习主线
00:00–02:35	问题定义：长期、持续、主动的协作
02:35–06:13	从复制粘贴到后台智能体
06:13–10:06	模型能力与文件化记忆
10:06–18:07	生命周期、架构、可靠工作流
18:07–27:26	Harness、工具、helpers 与安全
27:26–34:26	缓存、压缩与记忆分层
34:26–45:17	训练飞轮、未来问题与学习建议

视频在讲者总结与致谢处结束，没有包含随后预告的现场问答。

1 从工具循环走向常驻智能体

1.1 问题的起点：模型能执行，不等于任务能持续

讲座从课程第一周的实验开始：为模型提供工具，把模型调用、工具执行与结果反馈放进一个循环。这个简单的运行框架称为 **harness**，它使模型不再只生成建议，而能修改文件、运行命令、依据结果继续行动。然而，若循环依附于用户的笔记本，合上电脑后进程就可能停止；模型的能力并没有自动解决运行环境的生命周期问题。

常驻智能体要把这一循环延伸到服务器或云端。讲者描述的目标包括：用户关机后继续工作，能够经历重启或崩溃，拥有独立的 Linux 电脑，并在自己的桌面、浏览器与文件系统中完成任务。它可以保存文件，积累对用户工作方式的理解。长期任务的目标跨度也随之改变：不仅是几个小时、几天，而可能是几周、几个月。后者是讲者提出的愿景，需要不同的系统架构支撑，不应理解为所有现有智能体都已达到的可靠运行时长。

这里的三个词各有侧重：**always-on** 强调用户离线后仍可运行；**persistent** 强调状态与经验能够持续积累；**proactive** 强调不必等到用户每次手动提示才行动。Slack 消息、邮件和其他事件都可能唤醒智能体。它像同事一样，在后台推进任务，有重要更新才回来汇报；主动性也包括知道何时保持安静。

补充解释。 一个持续启动的进程只满足运行条件的一部分。若进程仍不知道之前做了什么，或无法接受外部触发，它就难以承担长期工作。开场提出了这些需求；恢复、存储和调度的具体实现要在后面的架构讨论中寻找，不能仅凭开场定义推定。

Your week-one agent	An always-on agent
Runs in your terminal	Runs on a server, so it survives crashes, deploys, and a closed laptop
Uses your laptop	Has its own cloud computer: a small virtual machine with a desktop, browser, and files
Remembers only the messages list	Saves its state, memory, and skills, and summarizes old messages
Prints every model reply	Only messages you when it calls a send tool, so it can stay quiet
Starts when you type	Starts on events: your messages, schedules, Slack, phone calls, finished work
Does one task, then exits	Works for months with the same memory, across many conversations

图 1: 本地实验智能体与常驻智能体的职责对比（裁剪）[1]

1.2 开场代码：工具结果为什么要回到消息列表

00:21–00:42 的幻灯片用下面的简化代码回顾课程第一周。它没有展示具体 SDK；关键是每次调用后保存回复，再把工具结果追加到消息中，让下一轮模型依据实际执行结果继续判断。

```
messages = [system_prompt, user_message]

while True:
    reply = llm(messages, tools=TOOLS)
    messages.append(reply)
    if not reply.tool_calls:
        print(reply.text)
        break
    for call in reply.tool_calls:
        messages.append(run_tool(call))
```

Listing 1: 开场幻灯片中的模型工具循环

当模型没有工具调用时，示例打印文字并结束循环。关闭终端也会停止整个进程，所以这一结构还没有持久恢复、事件唤醒或独立电脑。后续章节正是在这个小核心外补齐这些能力。

[1]原视频字幕定位区间：00:00:44–00:02:06；选用画面：00:01:35。

1.3 四个阶段：人的角色怎样变化

讲者把开发者与 AI 的协作归纳为四个阶段。第一阶段是从 ChatGPT 复制代码到编辑器，人负责把文本转化成真实改动。第二阶段是终端编码智能体：模型借助简单 harness 修改文件、执行 shell 命令，获得实际推进工作的能力。第三阶段是为编码智能体构建专用应用，允许多个智能体并行执行，部分任务搬到云端。但人仍关注每个操作，逐条检查命令是否正确，成为监督瓶颈。

第四阶段是常驻后台智能体：它拥有电脑、记忆，能够在工作中积累技能；用户去睡觉或做其他工作后，它仍继续推进。这是讲者的产品演进框架，并非互斥的历史分类；复制粘贴、终端交互和专用应用仍可能同时存在。

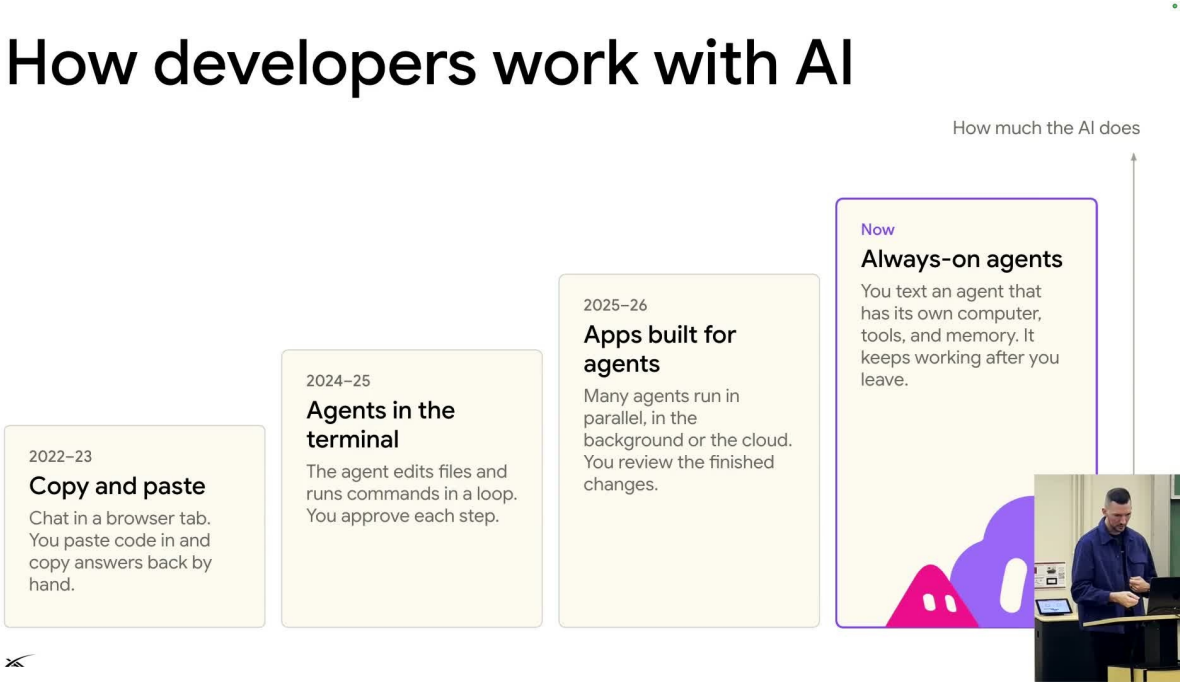


图 2: 开发者与 AI 协作的四个阶段[2]

1.4 从等待与批准转向持续引导

过去的交互是“发送提示、等待结果、再发送提示”；执行 shell 命令时，还可能需要批准或拒绝。常驻智能体则应允许用户在工作中补充条件。讲者给出的例子是先说“去看这个讨论串”，马上再说“顺便说一下，改成周四再做”。后一句调整的是前一个任务的执行时间。模型与 harness 需要识别两句话之间的联系，而不是将它们当成彼此无关的命令。

当用户要求发送邮件或 Slack 消息时，讲者介绍了 **auto-review**：由另一个 AI 模型检查准备执行的命令。他认为，人工连续审批数百条 shell 命令会产生疲劳，自动审查可能更有效；他还提到，在盲测中模型多数时候检查得更好。这些是讲者的判断，视频没有给出样本量、实验条件、评价指标或准确胜率。因此，教学上可以把它理解为一种分工：执行模型负责行动，审查模

[2]原视频字幕定位区间：00:02:39–00:04:04；选用画面：00:03:20。

型检查行动；不能据此宣称自动审查普遍优于所有人工审核。

1.5 聊天界面承载真实委托

讲者指出，模型能力提高后，界面反而可以简化。这种产品称作 bot，交互接近给朋友发消息。人们已经熟悉手机和短信，不必学习复杂控制台便能提出目标。变化的核心在结果：它不只是回答问题，而能接手较长任务，像将工作交给同事。

1.6 本章小结

常驻智能体是在模型工具循环之上，加入独立环境、长期状态、事件触发与持续引导。它既要在用户离线后推进工作，也要在用户改变要求时调整方向。讲座的关注点由“能否执行一次”转向“能否长期协作”。

2 支撑长期协作的能力与文件体系

2.1 为什么现在可以尝试这种架构

原幻灯片将模型与系统能力整理成六项：长期工作、委派助手、真实工具、电脑操作、文件化知识，以及从演示中学习可复用做法。

1. **长期指令遵循**。过去一两年，模型更能在较长时间内遵循指令、不混淆任务。配合适当的 compression 与 compaction，讲者认为模型可以运行几个小时甚至几天。上下文压缩的具体方法将在后文展开。

2. **子智能体委派**。主智能体的工作记忆最终会填满。它可以把任务交给拥有新上下文的助手，即 sub-agent；助手执行消耗大量 token 或工具调用的工作，再向主智能体报告进展。讲者称这带来“实际上无限的上下文”。准确的教学理解是将工作分布到多个上下文窗口，而不是单个窗口长度变成无限。

3. **更可靠的工具调用**。讲者回顾几年前模型工具调用不佳、容易产生幻觉的情况，并认为如今有了明显改善。其“幻觉基本解决”的措辞属于个人判断；视频没有给出可以支持普遍结论的评估数据。

4. **电脑操作**。除 shell 和文件外，模型可以拥有完整 Linux 电脑、制作演示视频，像人一样在屏幕上点击。这被称作 computer use。没有提供 API 或 MCP server 的 SaaS 产品和网站，也可通过界面使用。这里重点是无需人手完成界面操作，而不是声称这些应用没有图形界面。

5. **文件存储与检索**。信息写入文件后，模型可以检索、召回，再将相关内容带回当前任务。

6. **复用工作经验**。向 bot 展示如何完成工作，它可将做法保存成 skills、memories 或 routines，以后再次使用。

补充解释。 本段最后一项讲的是保存可复用经验，不是介绍模型权重的训练流程。文件中写入新的技能，与重新训练基础模型是两种不同机制；讲者在这一段没有提供训练方法或训练数据差异。

2.2 子智能体如何减轻主上下文负担

可以把主智能体的上下文理解为当前任务的工作台：它保留目标、约束和近期进展。如果大量搜索、阅读或工具输出都留在工作台上，空间最终会被占满。子智能体获得新的工作台，接手局部工作，再报告对主任务有用的结果。这一比喻帮助理解委派动机，但不意味着任务可以无限拆分而没有协调成本。

讲者在本段尚未说明子任务格式、结果压缩规则或并发数量。最终架构图应保留“委派、独立上下文、返回进展”这几个已有关系，避免补画未经讲述的实现细节。

2.3 四类文件解决四类问题

约在 08:30，讲者用一个简化例子解释文件如何承载长期记忆。首先是**用户档案**：保存对用户的理解，并加入每一次提示。例如用户偏好全小写、像短信一样的表达，这类稳定偏好应该影响每条消息。

其次是**工作日志**。它类似同事每天记录工作进展的笔记。日志不断积累，模型可以决定何时搜索。如果用户问“我上周做了什么”，它就检索过去的记录。讲者强调模型善于使用 Linux 命令、shell、grep 等搜索文件。这提供了一种直接的外部记忆方案，本段并没有要求使用向量数据库，也没有排除其他检索方式。

第三类是**技能**。技能把完成特定任务的方法编码为 Markdown 文件，包括具体步骤，以及哪些条件意味着成功或失败。讲者列举写好邮件、构建良好 API 等例子，并提到已有公司与开发者共同编写技能的生态。技能表达的是可重复使用的操作知识，不应与用户个人历史混成同一种记忆。

第四类是**例行任务**，即 routines。熟悉 cron job 的读者可以把它理解为按某种计划运行一段提示。技能回答“如何做”，例行任务回答“何时启动”；视频没有给出具体运行周期或实例。

用户档案与工作日志也具有不同的访问方式：档案是默认随提示携带的背景，日志是按需检索的历史材料。不能因为它们都保存为文件，就把全部历史每次完整放进模型上下文。

画面补充（08:34–10:02）。 原页给出的目录示例更具体：`memory/profile.md` 随提示带入；`log/2026-10.md` 按日期记录事实；`skills/linkedin-sourcing/SKILL.md` 与 `skills/weekly-metrics/SKILL.md` 按需加载；`routines/morning-brief` 由提示及“工作日 8:00”构成。它展示的是一个简化文件布局，说明例行任务同时包含“做什么”和“何时做”。

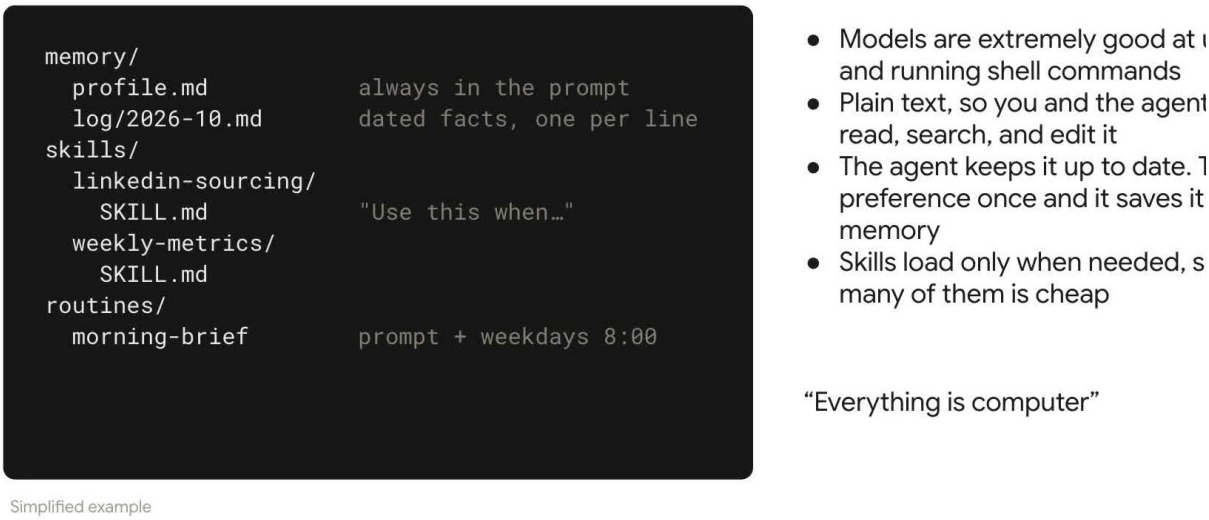


图 3: 用户档案、日期日志、技能与例行任务的文件示例（裁剪）[3]

2.4 插件连接工具与知识

讲者还提到 bot 产品中的插件体系：可以安装插件接入想用的工具，插件使用技能或服务器。本段没有交代插件格式、权限模型或安装步骤。原始字幕将产品名转录为 Grokbot，需要结合声音或幻灯片核实；理解机制时，重点在于插件把工具接入与可复用流程组合起来。

2.5 本章小结

长期协作依赖多个层面共同作用：模型能持续遵循指令，子智能体承接局部工作，工具与电脑让它实际执行，文件则保存偏好、历史和方法。用户档案、工作日志、技能和例行任务分别服务于稳定背景、历史召回、操作复用与定时触发。它们把“一次对话”延伸为可以积累经验的工作关系。

[3]原视频字幕定位区间：00:08:34–00:10:02；选用画面：00:09:10。

3 常驻系统的架构：事件、状态与可靠执行

3.1 持续可用为何不需要持续占用计算机

当一个智能体能隔天继续昨天的工作，又能在新邮件到来时主动行动，我们容易想象有一台计算机始终替它运行。讲者从资源生命周期解释这种体验如何实现：服务端和智能体计算机开始都可以处于睡眠状态；消息或其他触发器到达后，系统加载智能体状态，启动服务端执行。只有任务需要浏览器或桌面操作时，才启动供智能体使用的计算机。工作结束后，又可以让这些资源休眠。

这种设计的动机十分具体。若给每个用户安排一台全天运行的计算机，无论有没有任务都占用资源，费用和容量都难以承受。服务端执行同样应避免无事可做时持续耗费算力。常驻产品需要保存的是连续的身份与工作状态，计算可以随着任务出现而启动。讲者提到的持久状态包括对话日志、记忆和例行任务，它们保存在数据库等存储中，下一次唤醒时重新加载。

持续服务的基础是可恢复的状态

事件让执行开始，持久状态让前后两次执行属于同一个智能体。用户看到的连续性，由对话、记忆与任务的恢复共同形成。服务端执行和工具计算机则可以各自按需启动、休眠。

画面补充（10:14–11:13）。 生命周期图将服务端进程描述为“加载状态、工作、保存每一步”，并标注示例进程在空闲两分钟后停止。8:00 的 routine 可以再次唤醒它；若例行任务不需要电脑，电脑继续休眠。两分钟和 8:00 是该页的具体示例，不能推广成所有产品的固定规则。

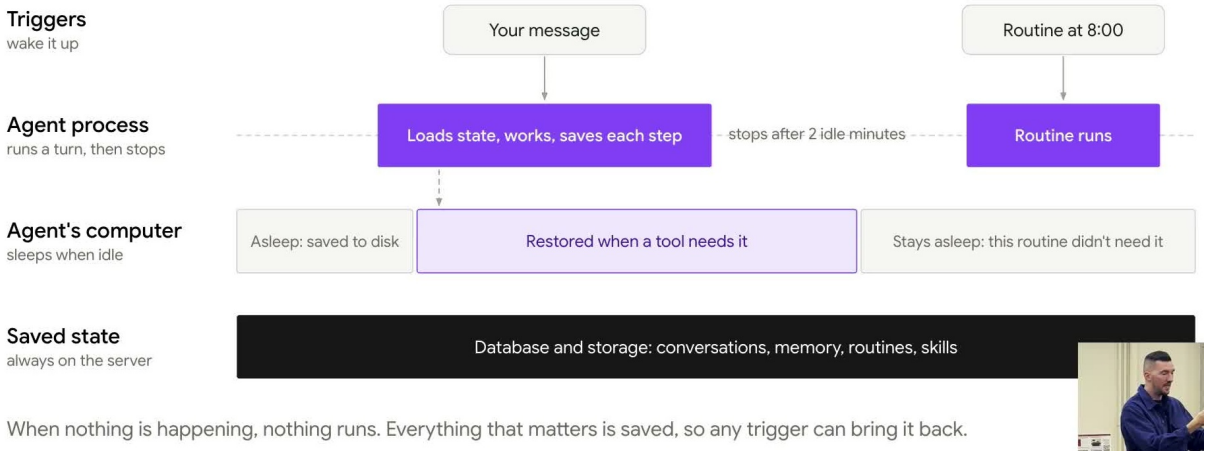


图 4: 事件触发执行、工具按需唤醒与持久状态（裁剪）[4]

3.2 事件入口、服务端与工具计算机各做什么

讲者把系统分为三个部分。左侧是事件入口，包括应用、Slack 消息和电子邮件；中间是服务端，接收命令并放入队列，随后进入智能体循环；右侧是智能体可以使用的计算机，提供文

[4]原视频字幕定位区间：00:10:14–00:11:13；选用画面：00:10:50。

件、命令、浏览器和桌面。智能体循环的基本动作很简单：模型判断下一步，调用工具，读取结果，再判断下一步。停止执行时，把所需状态保存到数据库或存储。

这个划分有助于理解产品复杂度来自哪里。模型调用工具的循环是核心，但单靠循环无法形成可靠的长期助手。外部事件必须能够到达它，工作必须能够保存，浏览器和命令必须有执行环境，用户必须能够看到结果。工具和服务连接围绕这个小核心逐步搭起来，才构成讲者描述的完整产品。

智能体可以由用户直接提示唤醒，也可以被后台事件唤醒。例如有人在 Slack 联系它，或让它参加电话和会议。讲者提到 SpaceX 员工无法参加会议时，会让 bot 代为参加、记笔记，再带回结果。另一类触发来自长任务：后台工作运行数小时甚至数天，完成后需要回到原会话汇报。

事件进来后，还有一个路由层负责去重，并决定如何把内容放进会话。随后，系统需要判断这件事是否值得打扰用户。讲者用同事解释这个取舍：如果同事每取得一点进展都跑来报告，反而妨碍工作；有的任务一天一次状态更新就够了，频率应随任务而变。因此，主动能力同时包含自行启动和有选择地通知。

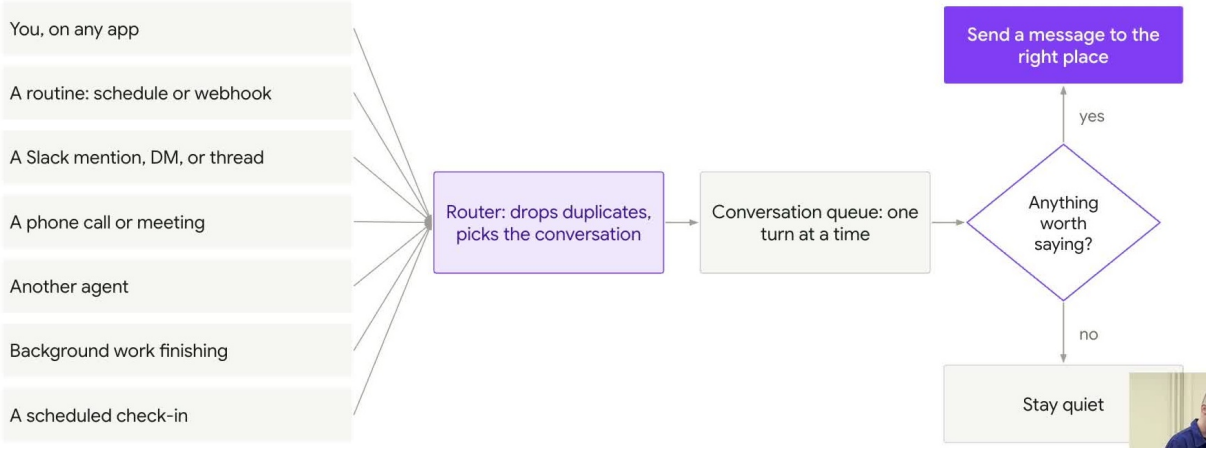


图 5: 不同事件经路由、去重与通知判断进入协作（裁剪）[5]

[5]原视频字幕定位区间：00:11:14–00:12:18；选用画面：00:11:45。

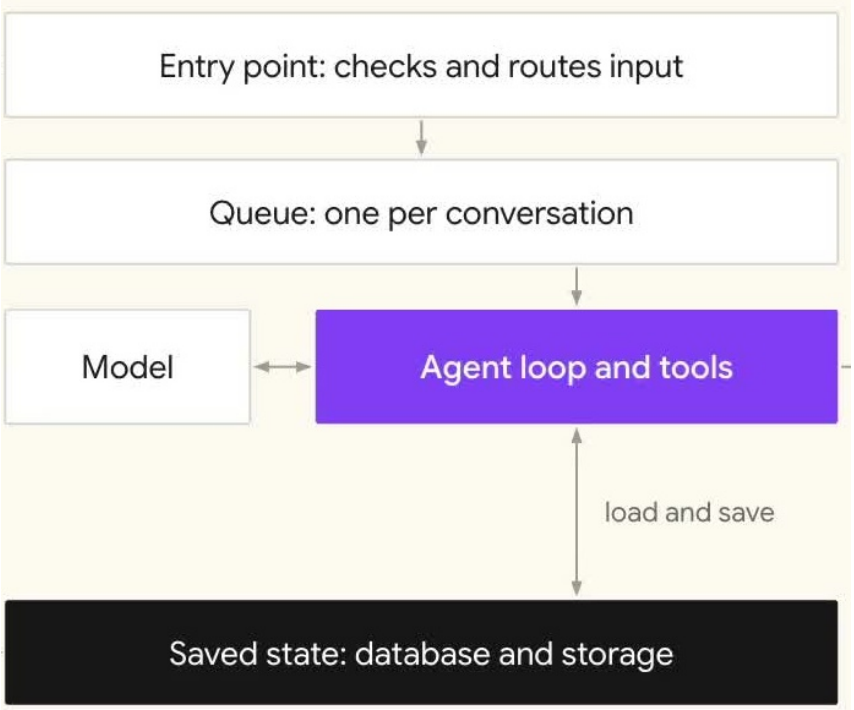


图 6: 三层总架构中的服务端：入口、会话队列、模型循环与持久存储（局部裁剪）[6]

3.3 Firecracker 计算机与独立生命周期

智能体的工具计算机是一个运行 Firecracker 虚拟机的 Linux 环境。讲者说明，虚拟机状态易于制作快照，因此能关闭机器，再恢复启动。该环境可以保存文件、执行命令、运行 Chrome。用户还可以连接进去，实时查看机器并接管操作。把命令放在独立、受限制的环境执行，也避免将工作直接压到个人笔记本上。

当多个 bot 都需要浏览器时，可以为它们提供不同虚拟桌面或屏幕。讲者将这些能力称为共享基础设施。这里的共享指供多个 bot 使用的基础设施能力，讲座在此处没有说明它们是否共享文件、登录会话或凭据，读者不应据此推断。

为什么智能体循环放在服务端，而非全部塞进这台 Linux 计算机？讲者称其为薄客户端、集中服务端的架构。这样修改智能体代码时，可以更新服务端，减少等待应用商店发布；转录、图像生成等较重计算也可能在服务端更快，具体取决于客户端设备；服务端还有成熟的扩容方法，能应对流量变化。

最重要的理由是独立管理生命周期。如果循环与工具计算机捆在一起，就难以单独关闭计算机。讲者给出的例子是 Linux 虚拟机需要紧急修补零日漏洞：若状态和调度逻辑保留在服务端，可以重建工具计算机，避免连带摧毁整个智能体循环。这个例子说明，计算机是可维护、可替换的执行资源，长期任务不能只依赖其当前内存。

[6]原视频字幕定位区间：00:13:16–00:14:05；选用画面：00:13:35。

补充理解：两种状态需要分别考虑

对话和任务进度属于智能体的逻辑状态；浏览器、文件与运行程序属于工具环境的状态。讲者分别描述数据库持久化和虚拟机快照。两者帮助恢复不同层面的连续性，不能把一次 VM 快照理解为自动保存了整个产品的所有状态。

画面补充（12:20–13:14）。 计算机页明确画出每位用户一台 Firecracker VM，内部是由 supervisor 保持运行的 Debian Linux 容器。其文件服务、命令执行器和虚拟屏幕分别承担文件连接、shell/截图/点击和 Chrome 操作；每个活跃 agent 可有自己的屏幕。休眠发生于最后活动几分钟后，避开 turn 中途：先同步文件到云存储，再保存内存和磁盘快照，最后停止机器。后续工具调用再恢复快照。页面同时说明安装工具和浏览器登录可以跨会话保留；用户的多个 bot 利用共享基础设施，并不意味着不同用户共用同一登录身份。

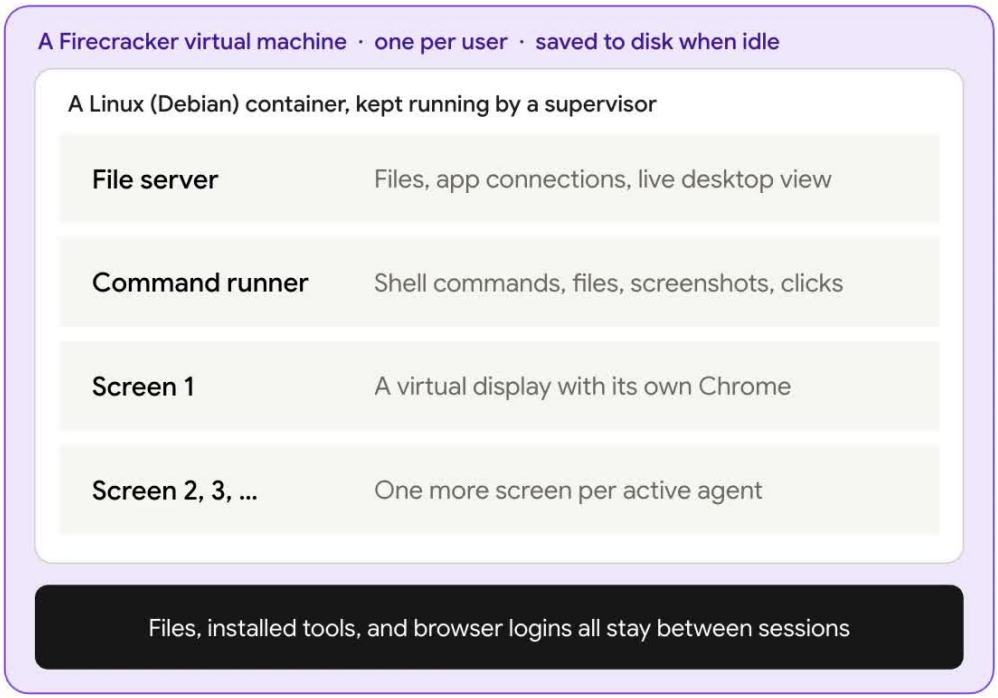


图 7: Firecracker VM 内部的 Linux 容器、执行器与虚拟屏幕（局部裁剪）[7]

3.4 发送消息之后：同步与去重

用户点击发送后，希望客户端立刻确认消息已经进入会话。服务端随后执行这一轮任务，让模型和工具循环工作，并将结果保存到数据库。客户端订阅服务端的变化信号，收到通知后，再拉取最新信息。这种安排使客户端不必承载核心执行逻辑，却能跟随后台工作更新界面。

网络和基础设施可能导致重试，同一条消息如果被重复处理或重复展示，就会破坏会话体验。因此需要去重。讲者还提到强一致性与最终一致性，作为读者可继续研究的分布式系统主题，但没有在这里给出具体同步协议或一致性保证。

[7]原视频字幕定位区间：00:12:20–00:13:14；选用画面：00:12:50。

3.5 从任务队列走向持久工作流

长任务真正困难的地方，常常出现在执行到一半之后。设想智能体需要完成步骤一、二、三，前两步已经完成，第三步遇到基础设施故障。如果系统只会把整个任务重新交给执行者，恢复时就可能从第一步开始重做。对于耗时操作，这会浪费工作；对于发送消息等带外部影响的操作，重复执行还可能产生实际问题。

讲者用这个例子区分简单任务队列与持久工作流（durable workflow）。持久工作流保存足够的执行信息，使任务失败后可以重试并延续进度。用户看到的效果是：第三步中断后，恢复时能继续第三步，前面的成果不必全部丢掉。产品使用开源 Temporal 承担这类基础设施能力，避免自己从头重造复杂的恢复系统，不过团队仍需要运行相关基础设施。

补充工程边界：恢复进度不等于副作用只发生一次

讲者是在比较简单队列示例与有进度恢复的工作流。队列系统也可以由应用自行保存进度；持久工作流则不能仅凭名称就保证付款、发信等外部行为绝不重复。处理此类副作用还需要相应的操作约束。本讲座在此处没有展开具体实现。

画面补充（16:18–17:20）。 恢复页明确强调安全重试需要幂等步骤：同一步执行两次也不产生额外影响。它同时列出 Temporal 这类引擎提供的定时器、取消及心跳。由此可见，恢复进度和安全重试共同构成可靠执行；外部副作用仍须按相应语义设计。

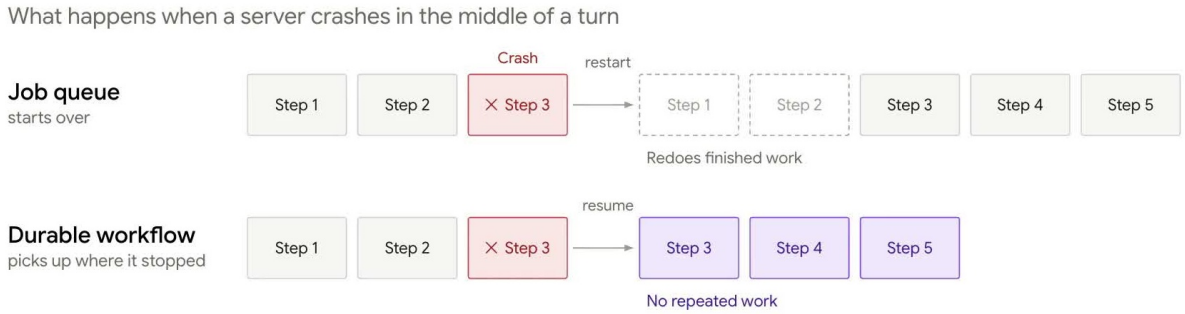


图 8: 步骤 3 故障后，从头重做与恢复进度的对比（局部裁剪）[8]

3.6 用户如何打断后台工作

长期助手必须允许用户改主意。用户直接给自己的智能体发消息，与群聊、Slack 线程或后台例行任务的优先级不同。讲者举例：用户突然说“其实把这个活动的日期改一下”，系统希望停止当前循环，优先处理这条新的用户指令。其他来源的事件通常进入队尾，等待后续处理。

短时间到来的三四条消息，也可以按启发式规则合并成一个 turn，减少不必要的重复启动，然后逐轮执行。这样，队列同时承担排序与合并功能，而持久工作流负责失败后的恢复。打断当前循环只表示改变后续执行方向；对于已经完成的外部操作，不能由此推断系统能够自动撤回。

[8] 原视频字幕定位区间：00:16:18–00:17:20；选用画面：00:16:40。

3.7 本章小结

常驻能力由事件入口、持久状态、服务端循环和按需计算机共同构成。事件决定何时启动，路由与去重决定内容如何进入会话，通知策略决定何时联系用户。持久工作流帮助长任务穿过故障，消息优先级让用户始终能够修正工作方向。理解这些分工之后，下一步才是看 harness 如何把模型、工具和用户交互连接起来。

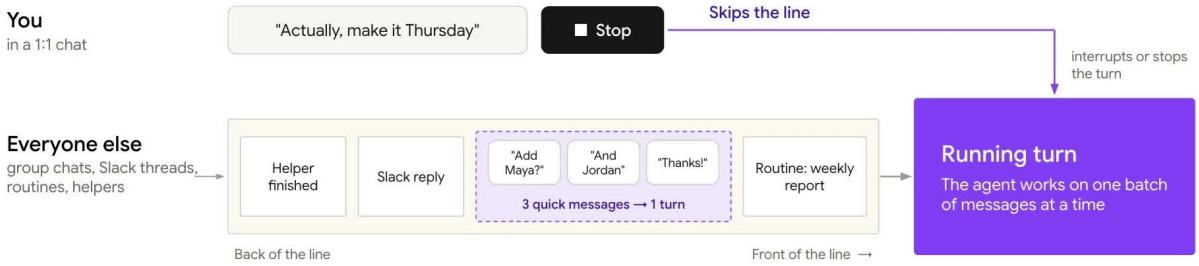


图 9: 直接用户指令优先、其他来源排队，以及快速消息合批（裁剪）[9]

4 Harness：工具发现、上下文分工与行动边界

4.1 一个沟通接口可以支持丰富交互

Harness 可以理解为模型周围的执行与协作框架：它规定模型能用什么工具，工具结果怎样进入任务，用户如何收到信息。讲者描述的 GrokBot 架构在客户端与服务端之间保留一个沟通工具，原幻灯片标为 `SendToUser`。客户端像给服务端发消息，运行 shell、读取文件、调用云端智能体、录屏与演示等重型能力放在服务端。

统一沟通方式便于扩展到移动应用、桌面应用、邮件和 Slack 等入口。每种客户端都能复用服务端能力，同时以适合自身场景的方式显示结果。单一接口也不限制界面的表现力：收到登录相关信息时，客户端可以显示专门表单并连接 1Password；支付场景可接入 Stripe Link、X Money 等服务；准备发送邮件或 Slack 消息时，可以展示可编辑、可审批的草稿；简单确认还可以只留下表情回应。

接口边界简洁，交互仍可以丰富

服务端负责执行与能力组合，客户端负责用户看得懂、操作得了的交互。审批草稿、登录表单和支付界面，都是把后台执行接回用户决策的方式。

[9] 原视频字幕定位区间：00:17:22–00:18:06；选用画面：00:17:45。

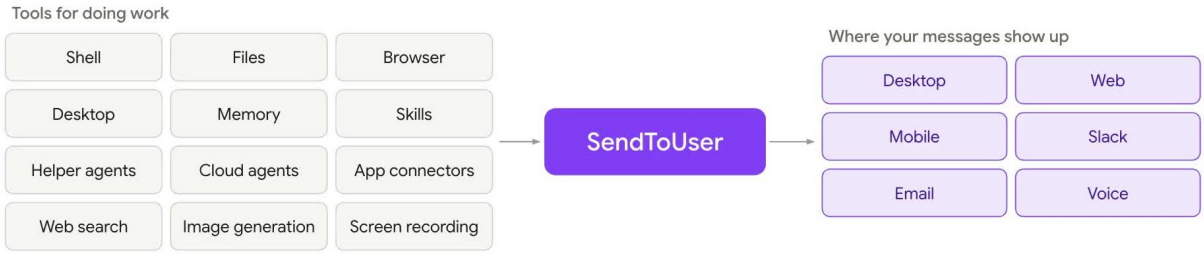


图 10: 丰富执行工具通过 SendToUser 连接不同消息渠道（局部裁剪）[10]

画面补充（18:08–19:14）。 这一接口还有四个作用：模型常规文本不直接展示给用户；没有新内容时可以不发消息；发出的内容可包含文件、选择题和安全密码输入；最后一次发送可以结束 turn，减少一次额外模型调用。因此，通知是显式工具行为，主模型在后台执行并不必然产生一连串聊天消息。19:14–19:59 的 UI 页进一步展示：密码只填入正在打开的网站，购买等待 Allow/Deny，邮件呈现完整草稿供 Send/Discard，简单反馈可用 emoji。

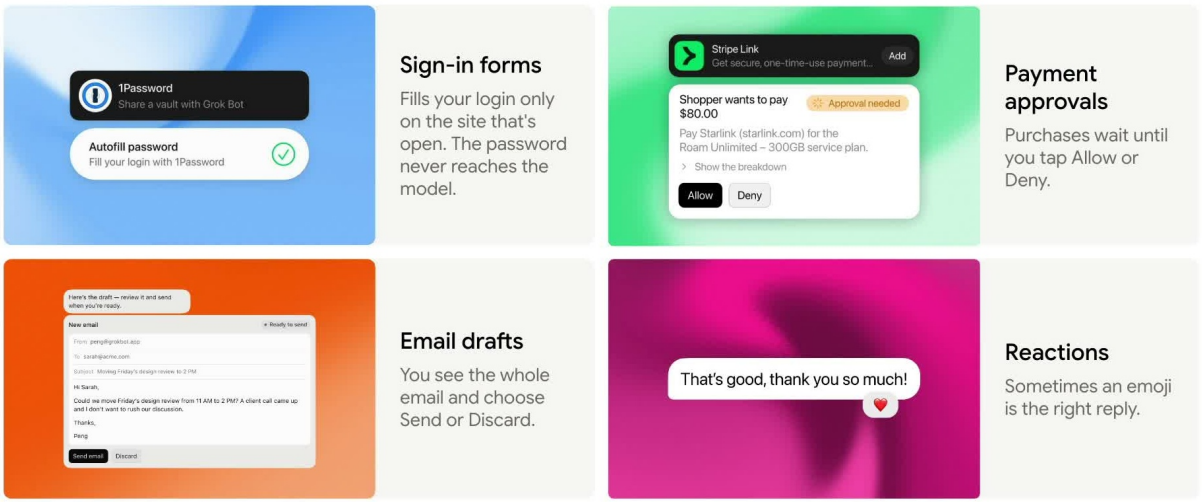


图 11: 密码填入、支付审批、邮件草稿与反应的专用交互（裁剪）[11]

4.2 动态工具发现：先索引，再读取细节

模型上下文有 token 上限。一个工具占用的不只是名称，还包括参数 schema、描述和各种使用细节。当用户连接 Google Drive、Gmail、YouTube 等 MCP 服务时，如果把所有工具定义都直接塞进每次请求，工具信息就会挤占会话与任务的空间。讲者用已被填满的上下文方块图说明这个问题。

动态工具发现把读取信息推迟到确实需要时。讲者描述的做法是：发给模型的消息里先放工具名称；模型决定使用某个工具后，再从文件读取详细信息。有些框架将这类能力称为 tool

[10] 原视频字幕定位区间：00:18:08–00:19:14；选用画面：00:18:45。

[11] 原视频字幕定位区间：00:19:14–00:19:59；选用画面：00:19:20。

search tool。文件和文件系统因此成为承载、读取工具说明的一种统一界面。结果是少占上下文、减少 token 使用，也降低相应费用。

这项机制解决的是工具说明过多的问题，并未让模型获得无限容量。它也不意味着系统可以省略参数语义：详细说明仍然要在调用前提供给需要它的模型。重要的是把完整信息放在正确时机，而不是每一轮都无差别携带全部信息。



图 12: 全量工具定义与按需读取细节占用的上下文示意[12]

4.3 先使用便宜而可靠的能力

工具很多之后，还需要选择顺序。讲者提出先尝试成本较低、可靠性较高的方式，再逐步增加操作复杂度。最简单的是使用会话历史中已有的信息；若还缺信息，再通过插件、MCP 服务或 API 获取，例如用 Plaid 访问银行信息。再下一层是网页搜索，例如查询昨晚 Cubs 比赛的比分。

如果这些途径无法满足任务，才进入计算机环境：打开浏览器访问网站，必要时使用完整桌面，运行脚本、命令或构建产物。前面的办法仍无法解决时，再请用户提供输入或批准操作。讲者的同事类比再次出现：一个有用的同事会先利用已掌握的信息和可用工具，再带着真正需要协助的问题回来。

求助顺序不能替代行动授权

先自行解决信息问题，可以减少打扰；发送邮件等不可逆操作，仍应遵守讲者随后强调的用户审批要求。两者针对不同问题：一个决定信息从哪里来，另一个决定谁有权让行动发

[12] 原视频字幕定位区间：00:20:00–00:21:30；选用画面：00:20:35。

生。

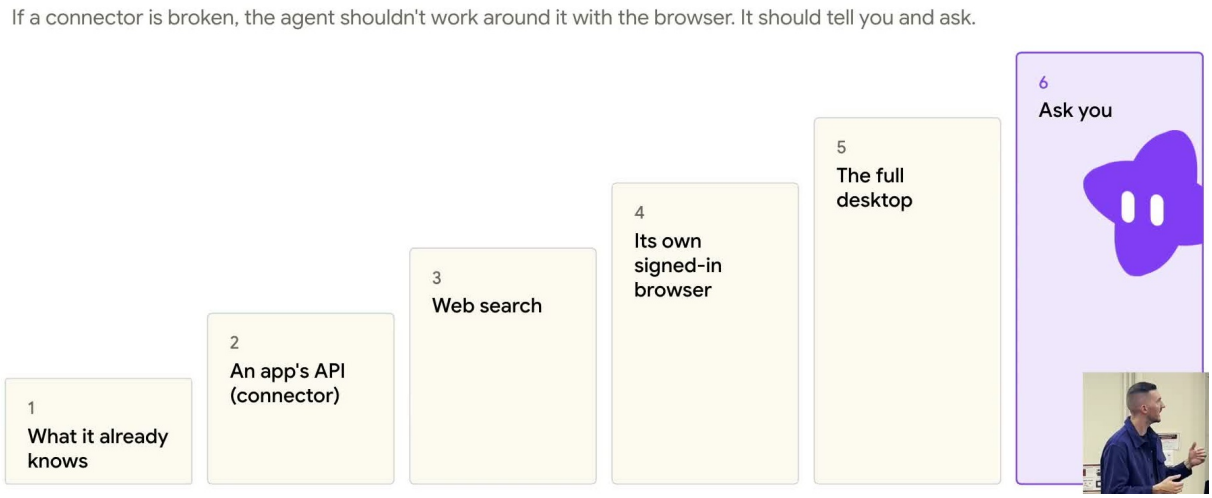


图 13: 从已知信息到 API、搜索、浏览器、桌面与用户求助的能力阶梯（裁剪）[13]

画面上的例外（21:31–22:35）。 能力阶梯上方还明确写道：若已有 connector 损坏，应告知用户并求助，不能悄悄改用浏览器绕过。这说明阶梯是合理选用能力的顺序，不是无条件寻找替代通道的规则。

4.4 理解网页：截图、HTML 与无障碍树

进入浏览器后，智能体并非只能看截图。讲者列出三种获取页面信息的途径：屏幕截图、HTML，以及无障碍树（accessibility tree）。无障碍树提供按钮、链接等交互元素的结构表示，能减少一些无关页面细节，帮助模型定位操作目标，并降低 token 使用。

这些方式需要互补。有的网站会出现弹窗或对话框，结构化读取不一定足够，此时仍要保留按像素点击的能力。二因素验证码或 CAPTCHA 则是需要用户介入的例子。讲者没有在此处声称某一种表示适用于所有网站，也没有提供特定浏览器框架的实现细节。

4.5 主智能体与 helpers：把执行细节留在局部

一个长期会话如果记录每次 shell 输出和每个浏览器中间状态，会很快被执行细节占满。讲者因此引入主智能体与 helpers。主智能体作为协调者（orchestrator），尽量保持会话稀疏；需要大量命令或昂贵工具调用的任务交给辅助智能体执行。辅助智能体在自己的上下文里完成工作，最终只把结果带回主智能体。

这里节约的不是任务本身所需的全部计算，而是主会话必须携带的中间信息。用户与主智能体可以持续讨论目标、结果和下一步，不必把所有辅助任务的操作过程摊在同一个窗口里。讲者说这样能让产品“感觉拥有无限上下文”，同时立即强调每个模型都有上下文限制。

[13] 原视频字幕定位区间：00:21:31–00:22:35；选用画面：00:22:05。

无限上下文是一种体验描述

局部执行与结果回传，可以缓解长会话中上下文膨胀的问题。每个模型的上下文仍然有限，结果整理也必须保留任务所需信息；不能把这种分工写成容量真的变成了无限。

不同 helper 有不同技能，而且各自是独立的持久工作流。例如视频 helper 因内存问题崩溃，可以恢复继续，避免丢失全部进度。主智能体管理待办列表，用户可以查询、发消息或停止 helper。这种结构借鉴团队处理复杂问题的方式：协调者保持任务全局，执行者掌握各自局部工作，再把结果交回来。本段没有进一步定义技能文件格式或插件协议。

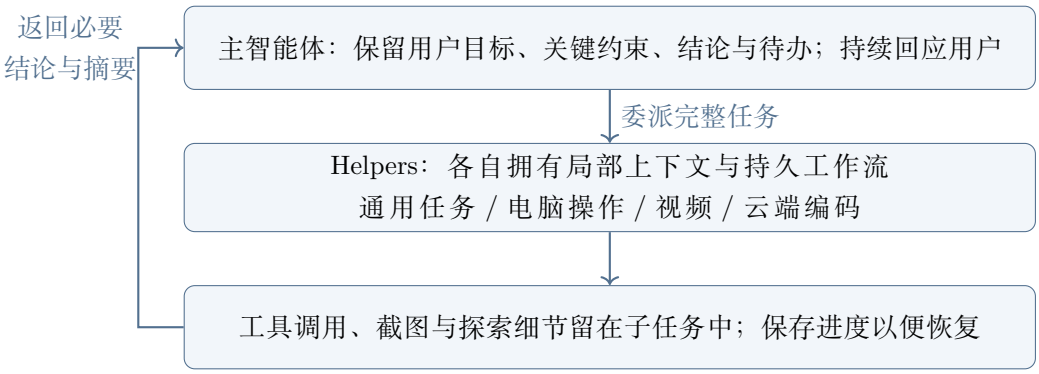


图 14: 主智能体与 helpers 的上下文分工（根据 23:31–25:12 整理重绘）

4.6 工具设计的三条实际经验

第一，工具说明需要控制长度。过长的描述与 schema 占据上下文，也增加价格与用量。讲者结合提示缓存（prompt caching）提出，尽量不要在 turns 中增加或删除工具。动态发现与稳定工具集合可以共同使用：基础沟通和读取能力保持稳定，需要的细节再按任务读取。这是对前后机制的整合理解，具体缓存策略未在本段展开。

第二，万能 shell 工具虽然能表达许多事情，但不一定是所有任务的最佳接口。讲者举例：如果模型有约九成时间都在用 shell 完成同一种操作，做一个专用工具可能更高效，也便于人检查产品到底做了什么。这里的九成是说明取舍的例子，并非基准测试数据。专用工具的价值在于把频繁操作表达得更直接，而非无限增加工具数量。

第三，错误消息要帮助下一步行动。给人看的好错误会告诉读者哪里失败、去哪个链接、下一步做什么；这些信息对智能体同样有帮助。讲者称这也是面向智能体的开发者体验（DX）。仅有“失败”两个字，往往不足以让模型正确恢复工作。

画面补充（25:13–26:28）。 工具设计页给出更具体的约束：每个工具描述和 schema 控制在 4,000 字符以内；暂时不可用的工具返回清晰错误，保持工具集合稳定。保存状态宜使用专用工具，便于审查者看清变化。示例错误是“计算机正在休眠；使用 Shell 或 Read 唤醒它，然后重试”。这里的字符预算属于讲座中的设计选择。

4.7 长期行动需要检查输入与授权边界

独立计算机帮助隔离用户机器，但长期助手还会接触私人信息并执行外部操作。讲者因此提出自动审查（auto review）：在 shell 命令执行前，让模型检查命令。此处给出的是机制方向，没有提供审查模型、具体政策或成功率，不能据此宣称所有危险操作都能被可靠识别。

另一条边界在输入来源。收件箱中的邮件可能是钓鱼内容，外部网络输入也可能夹带提示注入。系统不能把这些材料当成用户发出的指令。讲者明确说，行业仍需要在提示注入防护上继续工作。把外部内容交给模型分析，与让外部内容获得行动权限，应当分别处理。

对发送邮件等不可逆操作，讲者要求先询问用户。前面的草稿编辑与审批界面，正是将这一要求落到产品流程的方式。对于密码等敏感信息，还可以通过产品工作流让它们留在模型会话之外。讲座在这一段没有披露凭据存储或身份授权的具体架构，后续讨论应以原视频实际内容为准。

画面补充（26:30–27:25）。 安全页说明：审查模型阻止动作时，系统向用户请求审批，15 秒未答复仍保持阻止；邮件和 webhook 文本标为不可信，记忆被当成信息而非指令。页面要求新 routine 获得用户同意，在用户自己电脑上的动作也要审批。密码通过环境变量等工作流进入执行环境，保持在模型之外。这些是该产品在演讲中展示的边界，并非自动审查已经解决所有提示注入问题的证明。

4.8 本章小结

Harness 围绕模型循环组织执行和协作。单一客户端接口让多种入口共享服务端能力；动态工具发现减少每轮输入的工具负担；能力阶梯让系统优先采用更直接的办法；helpers 把大量中间执行信息留在局部上下文。工具描述、专用工具和错误信息决定执行效率，输入信任边界、命令审查和用户审批则决定长期代理如何取得并使用行动权限。

5 上下文工程：让长期运行成为可持续的系统

5.1 为什么常驻智能体必须管理上下文

当主智能体持续陪伴同一个用户时，每次调用都需要知道用户的要求、此前发生的事情和当前任务进展。对话不断增长，模型看到的材料也随之增加。因此，常驻智能体的工程问题包含两个方面：让重要信息始终可用，同时避免把所有历史无差别地重新放进每一次请求。

在 27:27–27:46，Lee 把上下文工程（context engineering）与运行框架工程（harness engineering）视为随时间演变、大体相通的说法。本段关注的是如何减少框架占用的上下文，以改善效率、成本与模型表现。理想的主智能体可以和用户保持非常长的对话，在后台多次压缩历史，而用户仍感到任务是连贯进行的。要实现这一点，就必须主动管理每次送入模型的信息。

核心理解

长期运行需要区分三个问题：哪些信息值得长期保存，哪些信息必须出现在当前上下文，以及哪些信息只在需要时查回。保存下来的资料，不必始终占用每次推理的输入空间。

5.2 提示缓存：复用稳定前缀，降低重复输入成本

普通多轮交互可以理解为：发送已有会话，再加一条新消息；下一轮又发送此前材料，再加新消息。这意味着很多输入 token 在相邻请求中反复出现。模型提供商通常对缓存输入给出较低的价格，所以长期会话中，理想情况下很大部分输入都应该能够复用缓存。讲者没有给出具体折扣比例。

如何提高缓存复用率？保持工具定义、系统提示以及其他固定信息尽量静态，并维持稳定顺序。把固定部分放在前面，把变化较频繁的材料与最新消息放在后面。这样，相邻请求共享的前缀更容易保持一致，新增消息构成主要增量。

缓存的作用是降低重复输入的处理成本。模型仍然需要面对历史信息的组织与取舍；缓存命中并不自动解决重要事实被忽略的问题。因此，提示缓存必须与后面的文件卸载、摘要和分层记忆一起使用。

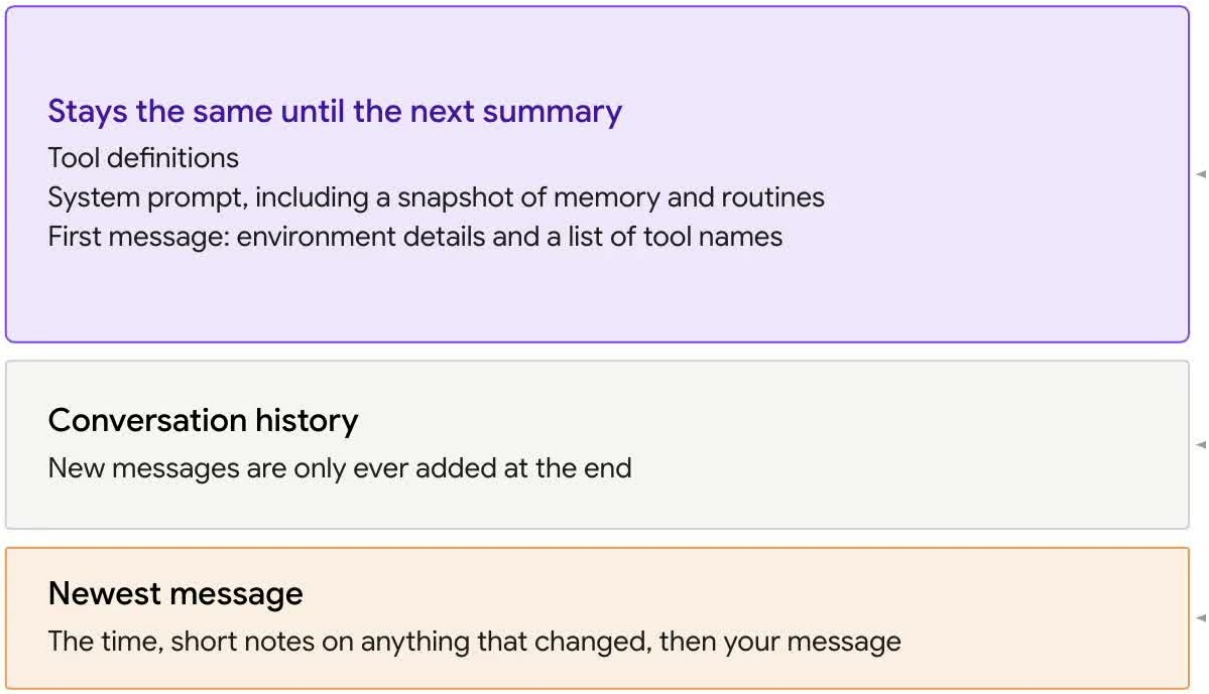


图 15: 稳定前缀、会话历史与新增消息的分层（局部裁剪）[14]

5.3 保护主智能体的上下文

Lee 给出三种具体做法。第一，工具产生很长的输出时，把结果写入文件，让模型需要时读取相关部分。文件在这里是智能体的外部工作空间，与读取一项技能或一条记忆相似。任务结果可以保存下来，而主会话无需始终携带全文。

第二，把截图等代价较高的材料交给 helper 或子智能体处理。子智能体负责消化大输入，主智能体获得推进任务所需的结果。这样，子任务的信息负担不会直接扩张主会话。这里的重点是保持主智能体的工作上下文紧凑；子智能体自己的输入和运行仍然会消耗资源。

第三，持续整理会话，修剪不再必要的内容。讲者用整理花园的比喻描述这种工作：随着任务推进，去掉无关材料，留下当前仍然重要的信息。这既影响成本，也影响召回和整体表现。

理解边界

把内容移到文件以后，仍需让智能体知道资料在哪里、何时应该查阅。上下文缩小只有在重要信息仍能被找到时，才真正有助于完成长期任务。

5.4 缓存顺序、发布变更与预热

持续运行会放大缓存失配的成本。讲者回顾 OpenClaw 在 2026 年 1 月（讲者所说的“今年一月”）流行起来时，许多人通过模型提供商的订阅方案运行它。这种全天工作的使用方式较新，

[14] 原视频字幕定位区间：00:27:47–00:28:46；选用画面：00:28:05。

运行框架尚未充分优化，出现较多缓存未命中，后续需要修复以尽量维持提示缓存。这里的时间应理解为相对于讲座发生时，不应依据读者阅读日期换成年份。

Lee 强调，意外的 cache miss 应当像系统缺陷一样受到关注。一个具体技巧是对提示的各个部分做类似哈希的管理，维持固定排列。部署新版本、试验新系统提示或加入工具时，都应检查是否改变了其他请求的稳定前缀。尤其在 A/B 测试中，不能让一组实验无意间打乱其他用户的提示顺序。

另一个技巧是缓存预热。当用户打开聊天窗口、开始输入时，服务端可以提前准备提示。讲者类比网页上的链接预取：鼠标悬停时，系统先准备下一页，而不必等点击之后才开始。提示顺序、缓存预热等小优化累积起来，影响长期 token 使用效率。

画面补充（29:57–31:32）。 缓存维护页把约束写得更明确：直到下一次摘要，工具、系统提示和第一条消息保持一致；部署改变工具描述时，运行中的会话接收变更便笺，而不是直接替换原有前缀。所有分段排序固定，各段单独计算哈希，便于定位是哪一部分改变。不可用工具返回错误，避免为单个 turn 增删工具。这些做法把“前缀稳定”变成可检查的工程行为。

5.5 上下文压缩：容量、成本与信息损失

即使不断优化，有限上下文窗口仍然会被长会话填满。Lee 假设模型具有 200,000 token 的窗口；持续追加信息，最终仍会到达需要摘要的时刻。他强调，摘要压缩会丢失信息，所以核心问题在于如何保留重要细节。减少篇幅本身比较容易，判断哪些事实不能丢失才是难点。

一个可操作的优化，是利用缓存仍然温热的空闲时段做压缩。假设用户和 bot 的会话已经超过 100,000 tokens，随后用户去做别的事情。模型提供商可能在一段时间内保留缓存，讲者以 10 分钟或 60 分钟作为示意。在这段时间内处理长历史，通常可以复用缓存输入；若等缓存失效以后再摘要，同样的长输入可能更贵。

因此，可以在用户离开后、缓存仍然可复用时提前产生摘要。用户约 30 分钟后回来，就从较短的摘要继续追加消息。这使后续输入规模更小。不过，成本下降与摘要质量必须同时考虑：若把当前目标、关键限制或任务状态压缩丢失，用户会感到系统忘记了正在做什么。

核心理解

压缩的目标是用较少的输入继续完成原来的任务。评价一份摘要，应检查它是否保留了后续行动需要的关键事实，而不能只看它缩短了多少 token。



图 16: 长会话、温热缓存、后台摘要与下一条消息的时间线（局部裁剪）[15]

画面补充（31:32–32:50）。 空闲压缩示意图还说明三点：若后台工作即将唤醒 agent，可以跳过此次压缩；面向用户的最后几条消息逐字保留；单个 turn 过长时也允许中途摘要。图中的“约十分钟后摘要”体现了利用温热缓存的时机，不能当成通用缓存保留时长。

5.6 分层记忆：稳定事实、历史日志与临时便笺

Lee 把记忆内容分成不同层次。核心事实应持续存在，例如用户要求输出始终使用小写。这样的偏好会直接影响每一条回复；如果某次突然出现大写，用户会察觉系统没有保持一致。

另一层是带日期的历史日志，记录过去发生的会话。日志提供可追溯资料，但不意味着每次必须把所有日期的记录全部带入。还有一层是 scratchpad，即当前工作的临时便笺。有些信息只在接下来 30 分钟或 1 小时内有用，任务阶段结束后即可清除。其他材料可以保存在文件里，模型通过运行命令和读取文件按需查找。

他用大脑作类比：正在讨论当前任务时，我们无需始终想着一场足球比赛。暂时不占据注意力的事情仍然可以留待以后想起。智能体也需要选择当下最相关的事实，并用类似垃圾回收的机制管理时效性。

背景与联系

本段的“垃圾回收”是一种类比：清理当前不再重要的内容，并管理事实的相关性。它不表示所有暂时用不到的长期资料都应永久删除。

[15] 原视频字幕定位区间：00:31:32–00:32:50；选用画面：00:32:20。

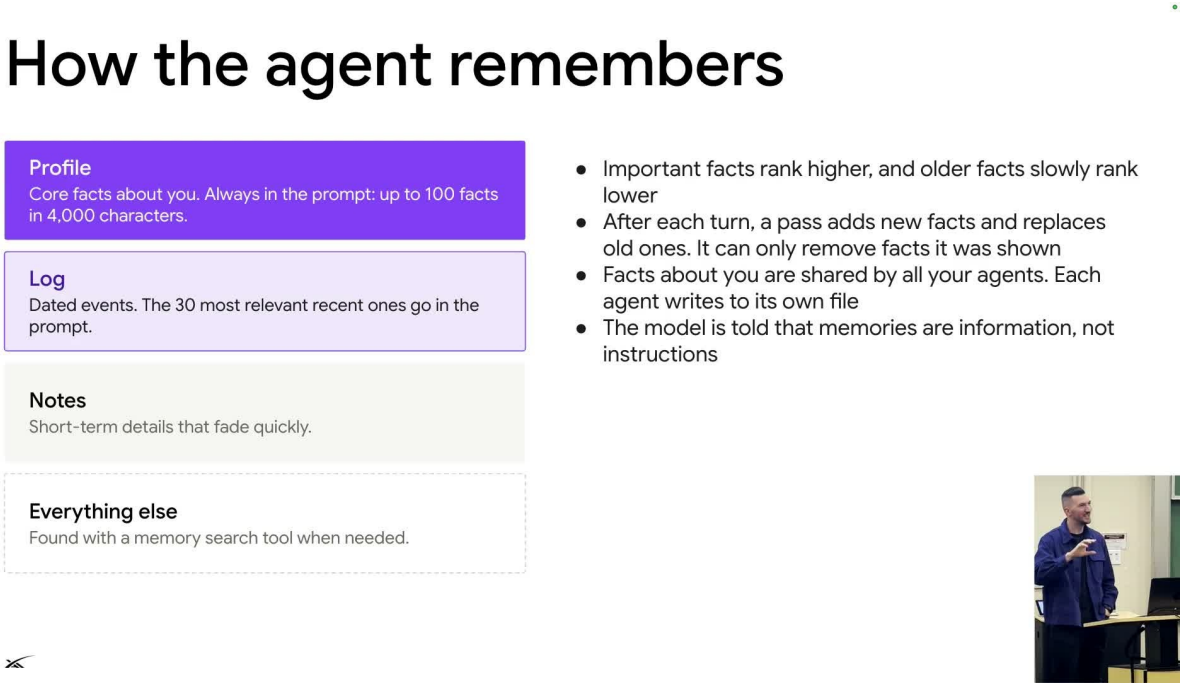


图 17: 核心档案、日志、临时便笺与按需搜索，以及事实更新规则[16]

画面补充（32:51–34:26）。 记忆页为四层给出实现示例：Profile 在最多 4,000 字符内保留至多 100 条核心事实；Log 选择最相关的 30 条近期事件带入提示；Notes 是快速淡出的短期细节；其余信息通过记忆搜索工具查回。重要事实排序更高，较旧事实逐渐降低；每个 turn 后更新事实，但只能删除本次实际见到的事实。关于用户的事实可由用户的多个 agent 共享，各 agent 写入自己的文件。所有这些数字与更新规则来自讲座页面，属于当时的设计示例。

5.7 本章小结

提示缓存降低重复输入成本，文件与子智能体减少主会话负担，摘要帮助应对有限窗口，分层记忆决定信息的保存方式与生命周期。常驻智能体需要把这些机制组合起来，让同一个任务在长时间跨度中既能延续，又能保持可承受的成本。

6 产品与模型如何一起进步

6.1 让模型熟悉自己的运行环境

有了工具和框架，还需要模型知道如何在其中行动。Lee 回顾三个训练阶段：预训练从大量互联网文本或其他私有数据中形成一般理解；监督微调（SFT）教模型按照助手或智能体的方式表现；强化学习（RL）通过反馈改善行为。他把最后一项类比为学习游戏并逐渐玩得更好。这是训练过程的高层介绍。

[16] 原视频字幕定位区间：00:32:51–00:34:26；选用画面：00:33:35。

对于常驻智能体，SFT 可以展示运行框架中的任务样例，使模型理解可用工具及其使用方式。RL 则可帮助改善指令遵循、选择正确技能，以及在浏览器中点击正确位置。由此可见，产品质量同时取决于模型训练、工具接口和运行环境。



图 18: 预训练、监督微调与强化学习如何适配智能体任务（裁剪）[17]

6.2 前台回应与后台工作的不同节奏

主智能体直接与用户交谈，低延迟非常重要。用户希望它迅速回应，所以框架和推理服务都可以围绕快速反馈优化。当工作交给 helper，且任务本来就需要很长时间时，优化目标可以有所变化。讲者举例：一个长任务运行 10 分钟还是 12 分钟，差别可能相对不大。

这一点为系统分工提供了依据：前台维持互动节奏，后台承担持续工作。视频没有规定某个部分必须使用某个模型，也没有给出具体推理预算；它强调的是，不同角色的等待成本不同。

6.3 内循环：把真实错误变成评测

假设智能体错误调用了一个不该调用的工具。理想做法是把这一失败转成 evaluation，让错误变成可以重新运行和检查的案例。随后定位问题，修复框架、程序缺陷、提示或推理环节，用评测确认改善，再发布变更。团队每天发现问题、修复问题、用 evals 测量，构成产品改进的内循环。

核心理解

可复用的评测案例把一次实际失败变成后续版本都能检查的质量约束。修复应同时回答两个问题：这次错误为什么发生，以及怎样检查同类行为是否改善。

6.4 外循环：训练新模型，解锁新产品行为

外循环的周期更长。每月、每几个月或其他周期训练的新模型，理想上会从之前积累的失败中学习，在产品关心的评测上提升。新版本进入产品后，还要确认这些能力确实改善了真实体验。能力跃升又可能带来新用法，产生新数据和新问题。

[17] 原视频字幕定位区间：00:34:27–00:35:35；选用画面：00:35:05。

群聊中的 bot 是讲者给出的例子。这类行为的训练数据相对少，多智能体之间怎样交谈、怎样避免不恰当行为，仍在探索。视频提及一些事件，但当前 ASR 中部分专名不清；这里保留其明确论点：新交互场景需要相应训练与评测。

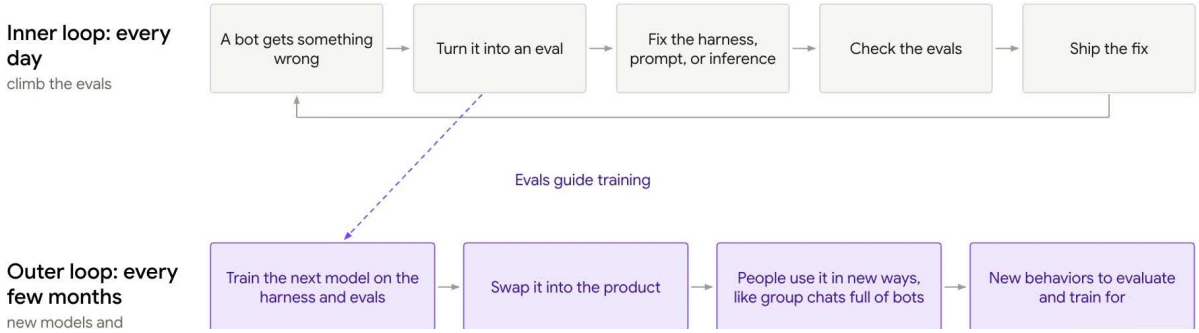


图 19: 日常产品修复内循环与模型训练外循环（裁剪）[18]

6.5 “Delete the product”：准备随着模型能力重构

Lee 说，在 Cursor 以及现在的 SpaceX，他们采用过 “delete the product” 的原则：假设今天的模型会是未来最弱的一代，系统就需要准备迎接能力变化。六个月后，界面可能需要完全重建；本次讲述的框架与上下文工程经验，相比六个月前乃至一年前已有明显变化。

这一原则鼓励面向下一代模型设计，尽量用少量构建提供简单接口，让模型替用户思考和行动。它表达的是愿意重新审视现有产品形态，而非实际删除代码的操作指令。越是围绕短期能力不足添加复杂结构，越需要在能力提升后检查这些结构是否仍然有价值。

6.6 本章小结

智能体产品有两个相互连接的改进节奏：日常内循环把失败转为评测并修复框架；更长的外循环让新模型学习这些经验，再进入产品。持续提升要求同时理解模型行为与系统机制，也要求产品能够适应模型能力的变化。

7 未来方向与开放问题

7.1 长期工作、记忆和技能学习

Lee 对未来 6–12 个月提出若干判断，应将它们理解为讲者预测。当时智能体可能工作数小时，或在某些任务中持续一天；他认为未来有可能持续数周、数月。运行时长增加，也会扩大训练和评测的难度。

他预计模型会更好地召回过去会话。底层记忆表示可能采用图、文件或其他形式，仍有多种研究路线。对于 “数字同事”，理想上可以像真人入职一样提供全部工作工具，不过组织仍在逐渐接受这一做法。

[18] 原视频字幕定位区间：00:36:06–00:37:44；选用画面：00:36:10。

交互可能进一步简化，一些用户会偏好直接用语音与智能体交谈。技能也可能在实际工作中学到，再编码进文件、指令和 skills，使通用智能适用于具体任务。同时，个人与组织需要更清楚地写下自身流程，让这些原本依赖隐性经验的步骤能够被智能体读取和执行。

7.2 长任务与评测周期的矛盾

一个尖锐问题是：如果评测一个智能体要让它运行一个月，而新模型也每月发布，该如何在发布前完成充分评测？真实任务长度与迭代周期可能发生冲突。Lee 提出两条方向：寻找更好的评测方法，或减慢发布速度。视频没有给出已经解决这一矛盾的算法。

此外，记忆算法、安全框架与基础设施仍有大量工作。运行环境需要避免被恶意参与者接管。这里保留讲者提出的问题，不把视频没有讨论的具体防御方案写成既定架构。

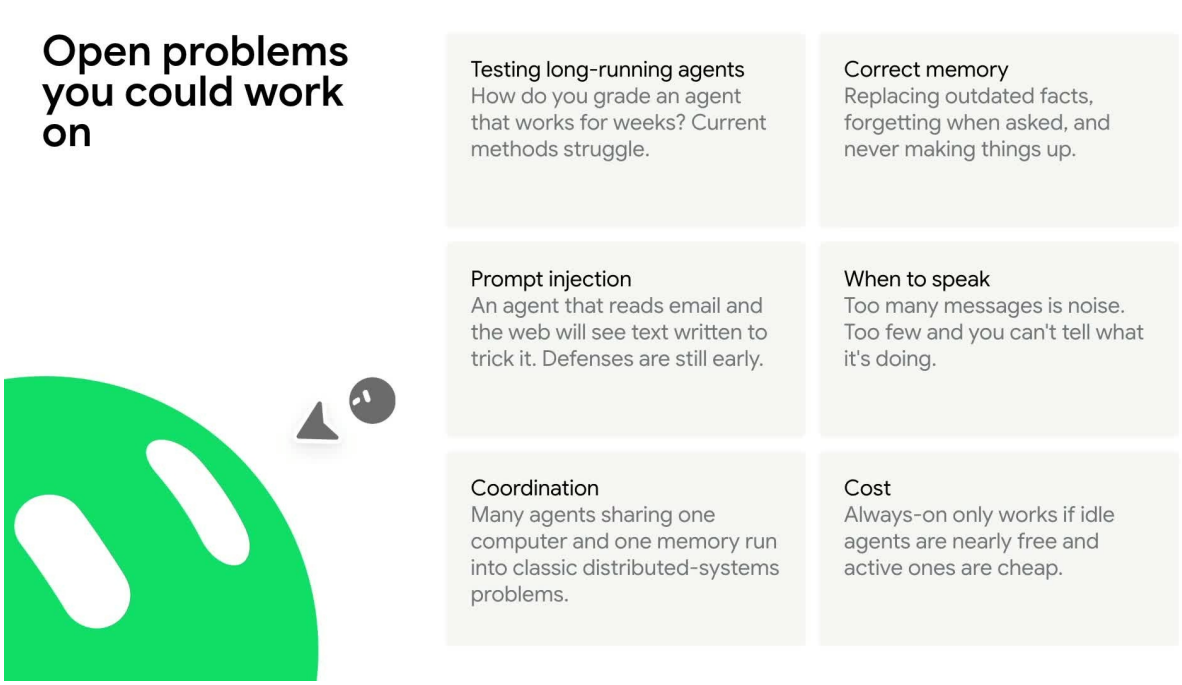


图 20: 长任务评测、记忆、提示注入、主动沟通、协调与成本六类开放问题[19]

画面补充（40:21–42:03）。 开放问题页还单列多 agent 协调：共享电脑和记忆会遇到经典分布式系统问题。正确记忆需要替换过时事实、在用户要求时遗忘，并避免编造。成本约束是让闲置 agent 接近零成本，活跃 agent 也足够便宜；这些是待解决的问题与目标，不是已达成的普遍指标。

7.3 主动性的分寸：什么时候应该打扰用户

常驻智能体可以观察事件，但什么时候应该出声，还没有完善的答案。Lee 举例：智能体发现用户昨天没有去上课，主动告诉用户自己获得了录音、整理了笔记，并把资料送来。这可能非

[19] 原视频字幕定位区间：00:40:21–00:42:03；选用画面：00:40:23。

常有帮助，也可能让用户感到烦扰。

主动性因此包含对时机与用户偏好的判断。系统既要避免错失有价值的提醒，也要知道何时沉默。这个问题与记忆直接相关：了解用户偏好、近期任务和已有状态，才更有条件判断某次提醒是否值得发送。最后这一联系是本笔记对讲座内容的归纳。

核心理解

主动型智能体的效果不只由“能否完成某件事”决定，还由它能否判断何时行动、何时告知用户、何时保持安静决定。

7.4 新的软件基础构件

Lee 预计全球 token 使用量会显著增加，因此推理、模型和框架的效率优化仍然重要。他转述一个比喻：今天许多 AI 应用都有侧栏、智能体和聊天框，就像早期 web 应用普遍包含服务器、数据库和 CRUD 界面。相似的构成反映了一组新的软件基础构件正在形成。

这些构件包括持久化工作流、沙箱、云电脑、记忆、技能、服务连接、语音、支付、身份、评测与可观测性。产品可能整合智能体能力，也可能成为被其他智能体调用的无界面数据组件。它们最终需要组合进运行框架，形成可用系统。讲者关于创业公司数量与投资额的口头估计不构成核实后的市场统计。

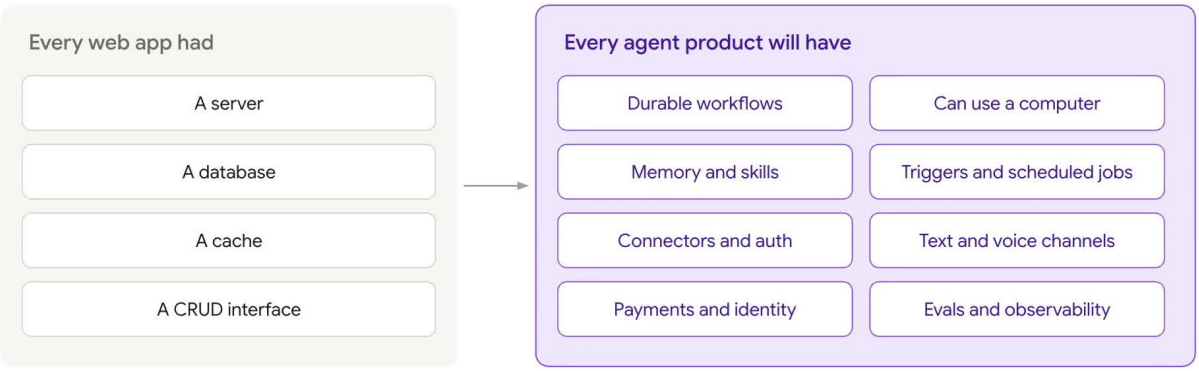


图 21: 传统 Web 应用与智能体产品的基础构件（裁剪）[20]

7.5 本章小结

更长的任务、更好的记忆、工作中学习技能和更自然的交互，为常驻智能体带来新的可能。与此同时，长任务评测、安全、主动性的分寸和 token 效率，决定这些能力能否稳定进入真实产品。这些问题与讲者最后的学习建议相连：理解架构、亲自构建，并学习模型工作原理。

[20] 原视频字幕定位区间：00:42:04–00:43:22；选用画面：00:42:08。

8 总结与延伸

8.1 回到问题：常驻、主动型智能体怎样工作

把讲座的机制连起来，可以得到一条完整的工作链。用户消息、外部事件或定时例行任务到来，系统路由并去重，恢复服务端状态，将任务送入智能体循环。模型根据当前目标选择工具；需要桌面时才唤醒云端计算机，需要大量探索时委派给有独立上下文和持久工作流的 helper。结果写入文件、日志和数据库，重要更新才通过客户端送达用户。执行结束后，计算资源可以休眠，任务状态和记忆继续保存。

三个词，对应三种系统能力

常驻依赖事件触发和可恢复运行，让服务在用户离线后仍可接续工作；**持续性**依赖持久状态、外部记忆与上下文压缩，让下一次执行接得上前一次；**主动性**依赖事件解释和通知判断，让系统在有价值的时候自行推进或联系用户。

下面的流程图根据讲座重绘，表达职责关系，不代表产品公开的完整部署拓扑。模型本身仍由推理服务提供，服务端循环负责组织调用。

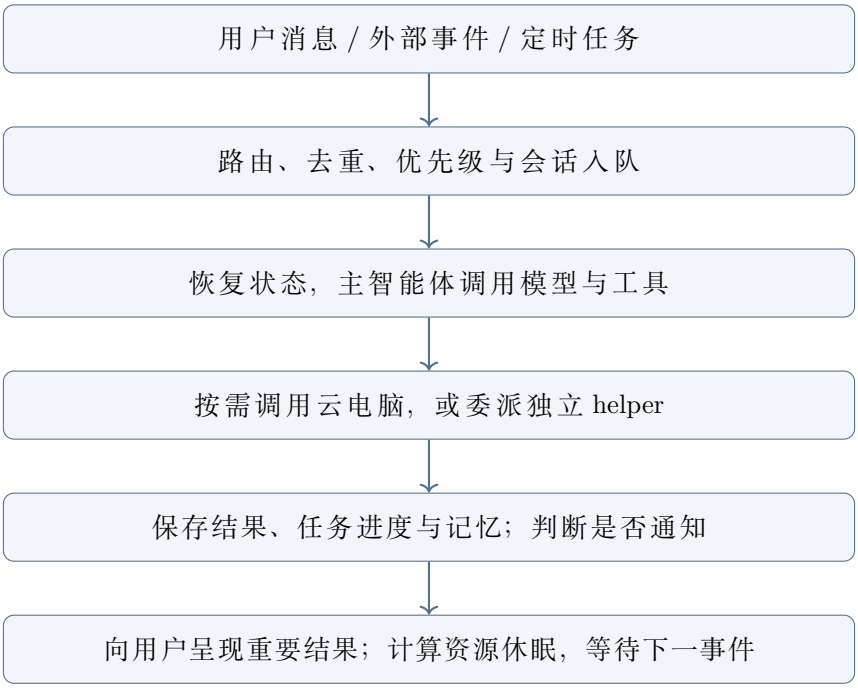


图 22: 从事件到连续协作的工作链（根据讲座整理）

8.2 用一个任务贯穿所有模块

补充教学例子。 假设你说“帮我跟进周四活动，准备材料，有变化告诉我”。这并非讲者的完整演示，而是把本讲机制组合成一个可理解的例子。系统先记录目标与时间。后台收到与活动相关的 Slack 消息后，路由器将其加入会话；模型先查已有记录或连接工具，而不是立即开浏览器。若需要访问只有网页界面的服务，才启动计算机。大量材料检索可以交给 helper，主会话保

留目标、关键结论与待办。你又说“活动改到周五”，这条直接指令应优先处理，调整后续工作。需要发信时呈现草稿并遵循审批要求；背景进度则由通知策略决定是否值得打扰。任务暂时告一段落，保存记录并休眠。几天后重新唤醒，系统依靠持久记录恢复工作，不要求你重新解释所有背景。

这个例子中，每个模块都有具体作用：工作流维持执行进度，记忆维持事实连续性，压缩维持上下文容量，缓存控制重复输入成本，工具选择控制行动成本，而用户引导维持目标正确性。省掉任何一层，都可能让系统在另一种时间尺度上失去连续性。

8.3 读懂一段最小教学伪代码

以下代码只表达讲座的职责划分，不是 Bot 的源码，也不是可直接运行的可靠执行实现。它刻意把“每次唤醒执行”与“持久状态存在”分开。

```
def on_event(event):
    turn = route_and_deduplicate(event)
    state = load_persistent_state(turn.agent_id)
    context = assemble_context(state, turn)
    result = run_agent_loop(context,
                            computer="on_demand",
                            helpers="durable_workflows")
    save_progress_memory_and_artifacts(result)
    if should_notify_user(result):
        present_result_to_user(result)
    release_idle_compute()
```

Listing 2: 一次事件唤醒的职责流程（教学伪代码）

真正的系统需要把可恢复执行放在持久工作流中，并让新用户指令能够引导当前执行；通知与审批也不能混为一项。伪代码的价值是帮助定位模块，不是替代前文的机制解释。

8.4 讲者最后给工程师的三条建议

讲者在 43:23–45:10 的收尾中首先强调**整体系统设计与代码架构**。架构决定会产生累积影响，因此他仍然阅读自己交付的代码，特别关注正在发布的系统结构。模型能写代码以后，工程师仍需要理解各模块如何协作、状态在哪里、生命周期如何分离。

第二条建议是**亲手搭建一个版本**。将本讲机制用到自己的智能体中，用沙箱、虚拟机和图形界面等组成一个可观察的系统。讲者口头提到可能存在开放实现，但视频未提供可确认的项目链接；学习目标是理解各部分的行为与关系，而非依赖一个未核实的名称。

第三条建议是**理解 LLM 的工作方式**。不一定要深入机器学习研究，但应有关于训练、工具使用和模型行为的直觉。正如构建 Web 应用需要了解数据库，构建智能体产品也需要理解模型，因为越来越多产品与 harness 决策受到模型能力影响。



图 23: 讲者的三条结尾建议：架构、亲手构建与模型基础（裁剪）[21]

8.5 进一步思考：在哪些地方衡量系统是否进步

笔记作者的综合。 可以沿三个维度判断一个常驻智能体的改进。第一是**任务连续性**：故障或休眠之后，能否接着正确的进度工作？第二是**认知连续性**：多次压缩后，目标、用户约束与关键事实是否仍可恢复？第三是**协作质量**：用户能否改变方向，系统是否在合适的时候通知，以及重要行动是否经过恰当的交互？这些维度对应前文不同模块，不能仅用一次回答的流畅程度代替。

长期任务与模型发布的时间尺度冲突，是讲座留下的关键问题。一个任务需要一个月，完整评测也可能需要一个月；当新模型每月发布时，怎样知道新版本更好，仍需有效评测设计。记忆算法、主动性的分寸和推理效率，同样需要持续观察真实行为，再将失败转化为评测与修复。

8.6 拓展阅读

- 原讲座：https://x.com/leerob/status/2108650243365736855。本笔记的事实、例子与时间轴以此视频为准。
- 持久工作流：https://docs.temporal.io/。
- 微型虚拟机：https://firecracker-microvm.github.io/。

8.7 本章小结

常驻主动型智能体的核心仍然是模型调用工具的循环。长期协作来自围绕这个循环建立的事件入口、持久工作流、独立计算机、子智能体、分层记忆与上下文管理。工程师的重点，是理解这些模块各自承担的职责，并通过真实失败、评测与系统设计让它们共同改进。

[21] 原视频字幕定位区间：00:43:23–00:45:10；选用画面：00:43:23。